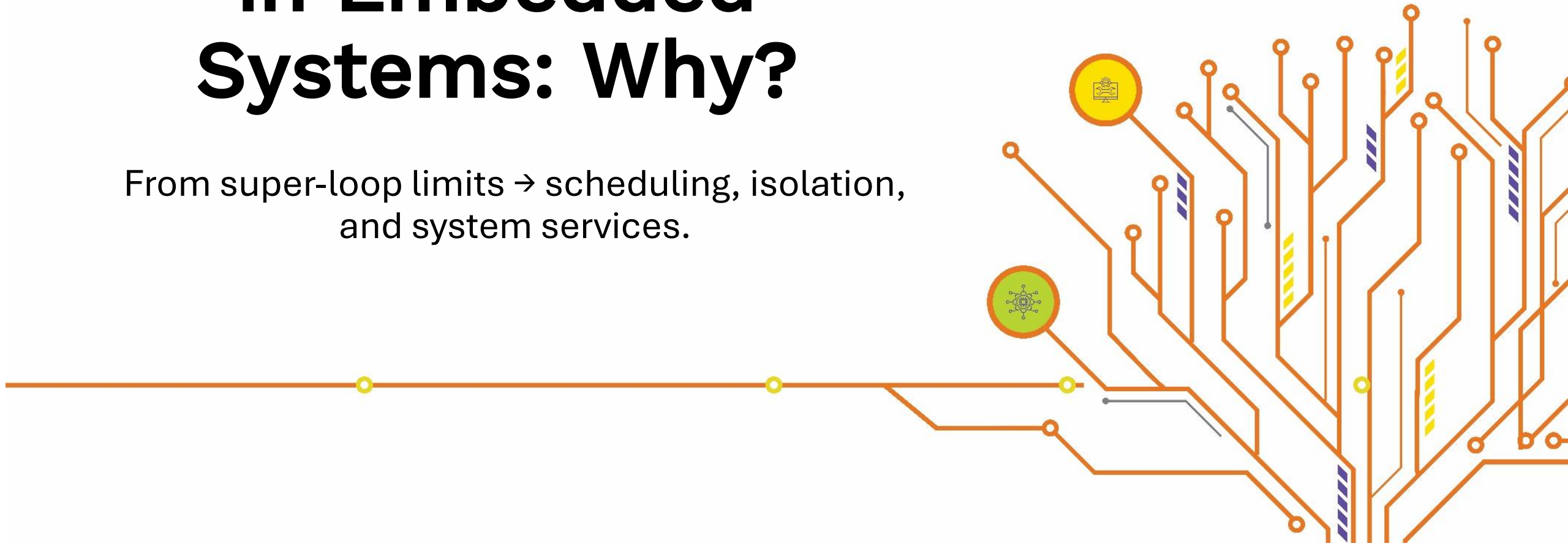
KPIT | APEX LAB

Operating System in Embedded Systems: Why?

From super-loop limits → scheduling, isolation, and system services.



Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

Users are advised to verify and confirm the correctness of the contents before use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

Problems Without an OS

|--read(5ms)--|--process(5ms)--|--display(3ms)--|--COMM BLOCK(200ms)--|

→ missed 50 ms sampling ticks

A - Concurrency & Scheduling

Multiple activities need CPU at different rates:

- Sensor sampling (50 ms)
- Communication (500 ms bursts)
- UI refresh (100 ms)
- Logging (every 60 s, but blocking)

Why this fails on baremetal:

- **Serialized loop** cannot respect **independent cadencaes**
- Blocking operations delay everything
- Interrupts help responsiveness but **not preemptive, priority-based scheduling**

Concurrency needs preemptive scheduling → an OS responsibility.

Problems Without an OS

B - Memory Protection & Isolation

Baremetal has:

- No process isolation
- No protection domains
- No crash containment

Result: one bug can corrupt the whole system (invalid pointer, buffer overflow).

Isolation requires **MMU + OS process model** (or MPU in small RTOS contexts).

With MMU (Linux) or MPU (some RTOS) can constrain damage:

- a fault kills one process or triggers a protection fault instead of corrupting unrelated code.
- MPU protects regions
- MMU provides full process isolation.

Problems Without an OS

C - IPC & Composition

- Features must talk reliably: sensor → control → comms → storage.

Without OS:

- Adhoc queues, globals, flags → Fragile, nonportable, non-observable

With an OS:

- standard IPC (queues, pipes, shared memory, sockets) makes flows explicit, testable, loggable.

- Isolation reduces blast radius; IPC makes interactions explicit & testable.

- Standard mechanisms: pipes, message queues, shared memory, sockets
- Easier to test, log, and maintain

Problems Without an OS

D - Filesystems & Persistence

Real products need:

- Logging to flash
- Configuration storage
- Durable updates

- VFS (Virtual Filesystem)
- Standard semantics
- Buffering and caching

E - Networking Stack

- TCP/IP, TLS, DHCP, DNS—these are **huge**.
- Baremetal approaches: Limited stacks, difficult integration, fragile edge cases

Linux provides:

- Mature network stacks
- Tools and libraries
- Sockets abstraction

F - Observability & Debugging

- Baremetal debugging is often limited.

- Logs (kernel/user)
- Process state (/proc)
- Trace tools (strace, perf)
- Profiling and metrics

Embedded OS vs Desktop OS

Embedded OS Priorities	Desktop OS Priorities
Determinism (predictable latencies)	Throughput
Fast boot	Feature richness
Low power	Multimedia / UI
Small footprint	Background services
Robustness (watchdogs, selective restart)	user session resilience

“*Embedded systems need predictable behavior under constraints.*

Linux can be tailored for embedded.”

Aspect	Bare-Metal	RTOS (e.g., FreeRTOS)	Embedded Linux
Execution model	Single program	Multiple tasks with priorities	Multiple processes
Responsiveness	ISR-driven	Deterministic scheduling	Preemptive scheduler (RT with PREEMPT_RT)
Memory protection	None	Often none (no MMU)	MMU-based process isolation
Footprint	Minimal	Small to medium	Large
Device abstraction	Manual	Manual	Provided by kernel drivers
Drivers / Filesystem / Networking	None (custom implementation)	Limited / optional	Mature and built-in
User space richness	None	Limited	Rich user space
Real-time capability	Very high (simple cases)	High and deterministic	Possible with PREEMPT_RT
System complexity suitability	Very simple devices	Mid-complexity real-time systems	Feature-rich systems

Choose the right tool based on **complexity + constraints + risk**.

When to Choose BareMetal

Characteristics:

- Hard real-time **with microsecond-level deadlines**
- Ultralow power, ultralow footprint
- Simple feature set; deterministic control loops
- No memory protection; single image; fast boot (milliseconds)

Baremetal excels when you **know exactly what must happen, when, and forever**, and the device has **one job**. You sacrifice isolation and rich features to get **predictability and speed**.

BareMetal: Use Cases

Motor Control / BLDC Commutation

- Deadline: PWM update and current sensing in tens of microseconds
- Determinism: Strict ISR timing
- Why baremetal: Minimal latency paths; tight control loop in ISR; resource-constrained MCU

Pacemaker / Implantable Medical Controller

- Deadline: Beat detection and pulse timing in milliseconds
- Determinism: Safety-critical, simple control logic
- Why baremetal: Highly constrained hardware; validated single-purpose behavior; rigorous certification

Capacitive Touch / LowPower Sensors

- Deadline: Scan cycles in hundreds of microseconds
- Why baremetal: Simple, event-driven, ultra-low energy

Keyless Entry FOB / LowDuty RF

- Deadline: RF timing windows in milliseconds
- Minimal feature set; fast wake → action → sleep

When to Choose an RTOS

Characteristics:

- Hard/firm real-time tasks with **bounded latency**
- Deterministic scheduling with priorities (SCHED-like internal policy)
- Typically, **no MMU** → limited memory isolation
- Moderate complexity; timer services; queues; semaphores
- Small footprint; faster boot than Linux
- Common pitfalls: priority inversion (fix with priority inheritance), timer drift (use monotonic clocks), ISR-only work (move to deferred tasks)

RTOS gives structure—tasks, priorities, timers—without full OS isolation or giant subsystems. Think **deterministic control with moderate complexity**.

RTOS: Use Cases

Quadcopter Flight Controller

- Deadlines: Sensor fusion loop (IMU/Gyro) 1–2 kHz, control outputs <1 ms jitter
- Why RTOS: Tasks for sensor read, attitude estimation, control output; priority-driven preemption; still lightweight

Industrial Servo / CNC Axis Controller

- Deadlines: Motion control **<1 ms jitter**
- Why RTOS: Deterministic scheduling + timers; modest networking via lightweight stacks; simpler than Linux, more structured than baremetal

Automotive ECU – Airbag Control

- Deadlines: Sensing/actuation **<1–5 ms**
- Why RTOS: Deterministic tasks, AUTOSARstyle scheduling, watchdog integration; no need for Linux's full user space

Smart Metering / LowTier Gateways

- Requirements: Meter readout tasks + modest comms (PLC/ZigBee) + small file logging
- Why RTOS: Balances real-time needs with modest I/O complexity; footprint constrained

When to Choose Embedded Linux

Characteristics:

- **Feature rich**: filesystems, networking, security, drivers, update systems
- **Process isolation** (MMU): one crash ≠ total device failure
- **Soft real-time** is typical; **hard real-time** possible with **PREEMPT_RT**, CPU isolation, mlockall, and tuned I/O
- Larger footprint; boot time generally longer (can be optimized)
- Excellent **observability** and maintenance story

Embedded Linux shines when you need **many moving parts**: networking, storage, UI, updates, security. If you also need **realtime**, you can combine Linux (for features) with **RT tuning** or pair it with an **MCU/RTOS** for the tightest loops.

Embedded Linux: Use Cases

Automotive Infotainment (IVI) / Digital Cluster

- Tasks: UI rendering, multimedia, navigation, connectivity, OTA updates
- Real-time: Soft real-time for UI responsiveness; some hard real-time delegated to dedicated MCUs (e.g., speedometer from CAN on separate controller)
- Why Linux: Complex feature set + isolation + security + vendor driver support

Industrial Gateway / Edge Compute

- Tasks: Protocol conversion (Modbus/OPC-UA), secure telemetry (MQTT/TLS), local analytics, fieldbus interfaces
- Real-time: Requires bounded latencies for field I/O; PREEMPT_RT + CPU affinity can deliver submillisecond jitter
- Why Linux: Networking, storage, containers, remote management; rich tooling

Home/Enterprise Router

- Tasks: NAT, firewall, WiFi, DHCP/DNS, VPN, captive portal, logging
- Real-time: Mostly **soft real-time**; high throughput over deterministic hard deadlines
- Why Linux: Mature network stack, drivers, security patches, extensibility

Autonomous Mobile Robots

- Tasks: SLAM, path planning, sensor fusion, vision pipelines, safety zones
- Real-time: Some control loops offloaded to microcontrollers; Linux runs **complex perception and planning** with tuned real-time threads
- Why Linux: Libraries (OpenCV, ROS2), IPC, logging, updates, security, driver availability

Comparative Thinking: Latency & Determinism

BareMetal	Fastest path to the hardware; µs–low ms	jitter minimal	You own everything; no isolation
RTOS	ms–sub-ms	bounded jitter	Priority preemption; limited isolation
Linux	low-ms typical	jitter controlled but non-zero	Rich features + isolation; needs **RT tuning** for strict deadlines;

Give a thought …

Application	Typical Thought	Correct Thought
Washing machine	Linux is overkill RTOS is enough Baremetal is fine	Baremetal or RTOS is sufficient; Linux likely overkill unless adding WiFi + rich UI + OTA updates.
Need TCP/IP + TLS + VPN and also 10 kHz sensor fusion	Linux can do both	• Linux can handle complex networking better, but 10 kHz fusion is often safer on MCU/RTOS due to jitter; • Linux can host higherlevel fusion/planning. • If kept on Linux, use PREEMPT_RT, pin threads to isolated cores, avoid page faults (mlockall), use poll/epoll, and avoid blocking I/O.
Need submillisecond jitter	Linux with tight control loop RTOS with custom update mechanism Two processor design	Split architecture: MCU + RTOS for motor control, Linux SBC for UI/network/updates.